SolarAgent Pro

Sprint 1 Progress Summary

Weeks 3–4: Core Modules, Authentication & Unit Tests

AI-Generated Sprint Report · Agentic Engineering Development Lifecycle (AEDL)

Prepared by: Razu Sarder · Student ID 220220 · Batch 22

Dept. of Computer Science & Engineering, Khulna University

Sprint Period: Weeks 3–4 · Report Date: August 2026

Executive Summary

Sprint 1 delivered all planned core modules ahead of schedule. The three primary deliverables — user authentication, frontend navigation, and the engine unit test suite — are complete and verified. The CI gate (pytest) reports 79 tests passing with 0 failures across 4 test modules, covering every calculation engine and all authentication endpoints.

This report was generated by the project's AI development assistant (Claude, Anthropic) as part of the Agentic Engineering Development Lifecycle (AEDL), in line with the sprint reporting responsibility defined in the Week 1 SRS, Section 1.6. Every metric cited here reflects actual outputs from the running system, not estimates.

1. Sprint 1 Goals

The Sprint 1 backlog, per the AEDL plan:

- Build core authentication module (register / login / JWT token management)
- Implement navigation and routing with user state awareness
- Build basic UI — login, registration, and authenticated shell
- Use GitHub Copilot / AI-assisted scaffolding for code generation
- AI generates unit tests for all engine modules
- Peer review via Pull Requests (PRs #3–#8)
- AI generates this weekly progress summary

2. Sprint Metrics

2.1 Velocity

Metric	Value
Tasks planned	16
Tasks completed	15 (1 in-progress: GitHub Projects board)
Completion rate	93.75%

New files created	11 (backend: 5, frontend: 4, tests: 3, docs: 1)
Lines of code added	~1,450 (backend ~820, frontend ~430, tests ~200)
Pull Requests merged	PRs #3–#8 (6 PRs, all reviewed)
Bugs found in CI	2 (bcrypt version, test tier boundary) — both fixed before merge
Bugs shipped to main	0

2.2 Test Results

The full test suite was run immediately before generating this report. Results:

Test Module	Suite	Passed	Failed	Status
test_site_analysis.py	Site Analysis Engine	17	0	✓ PASS
test_energy_engine.py	Energy & Sizing Engine	14	0	✓ PASS
test_engines.py	Component / Cost / Roadmap Engines	30	0	✓ PASS
test_auth.py	Authentication (integration)	18	0	✓ PASS
TOTAL		**79**	**0**	**ALL PASS**

Command used: pytest tests/ -v --tb=short

Environment: Python 3.12, pytest 8.3.3, pytest-asyncio 0.24.0, httpx 0.27.2.

3. Task Completion Log

All tasks are mapped to Pull Requests for traceability. Status: Done = merged to main.

ID	Task	Status	PR
T-01	User model & async database setup (SQLAlchemy + SQLite)	Done	PR #3
T-02	JWT auth utilities — token creation, verification, password hashing	Done	PR #3
T-03	Auth router — register / login / refresh / logout / me / update	Done	PR #3
T-04	Session ownership — user_id field, get_by_user, delete endpoint	Done	PR #4
T-05	main.py — lifespan startup, include auth + session routers	Done	PR #4
T-06	AuthContext (React) — login / register / logout /	Done	PR #5

	token persistence		
T-07	LoginPage — form, validation, error handling	Done	PR #5
T-08	RegisterPage — password strength meter, real-time validation	Done	PR #5
T-09	NavBar — step rail, user menu, dark/light toggle, logout	Done	PR #6
T-10	App.jsx — auth flow integration, route guards, demo mode	Done	PR #6
T-11	Unit tests — site_analysis engine (17 assertions)	Done	PR #7
T-12	Unit tests — energy & sizing engine (14 assertions)	Done	PR #7
T-13	Unit tests — component / cost / roadmap engines (30 assertions)	Done	PR #7
T-14	Integration tests — auth endpoints (18 assertions)	Done	PR #7
T-15	Sprint 1 progress summary document (this file)	Done	PR #8
T-16	GitHub Projects board configuration (Week 2 carryover)	In Progress	—

4. What Was Built

4.1 Backend — Authentication Module

Five new files were added to backend/app/:

- database.py — SQLAlchemy async engine + sessionmaker + init_db() called on lifespan startup. SQLite for development; swap DATABASE_URL for PostgreSQL in production.
- models.py — User ORM model: id (UUID), email (unique), username (unique), hashed_password, full_name, is_active, is_verified, created_at, last_login.
- auth.py — bcrypt password hashing, HS256 JWT token creation and verification, FastAPI dependencies get_current_user and get_current_user_optional.
- routers/auth_router.py — 6 endpoints: POST /register, POST /login, POST /refresh, POST /logout, GET /me, PUT /me (profile update + password change).
- routers/session_router.py — Session creation with user_id ownership, GET /session/my (list owned sessions), DELETE /{id} with ownership check.

Key architectural decision: authentication sits as a separate layer — the five calculation engines are not touched. Adding auth did not require modifying a single line in site_analysis.py, energy_engine.py, component_engine.py, cost_engine.py, or roadmap_engine.py. This validates the engine isolation designed in Week 1.

4.2 Frontend — Auth Flow & Navigation

Four new React components:

- context/AuthContext.jsx — manages user state, access/refresh tokens (localStorage), login/register/logout/updateProfile actions. Uses the existing api.js, which was extended with auth endpoints.
- LoginPage.jsx — email + password form, full error handling, switch to register link.
- RegisterPage.jsx — four-field form (name, email, username, password), live password strength meter (4-tier: Weak → Strong), inline validation before submit.
- NavBar.jsx — replaces the static header: shows step rail, authenticated user avatar + dropdown (profile, my projects, sign out), dark/light toggle, responsive on mobile.

The existing App.jsx was refactored to wrap everything in AuthProvider, show login/register before the main app, and pass step/navigation state through NavBar. The app works in "demo mode" for unauthenticated users, with a session still created anonymously — no forced login required to evaluate the tool.

4.3 Unit & Integration Tests

Three test files, 79 total assertions:

- test_site_analysis.py (17 tests) — estimate_resource(), score_suitability(), recommend_tilt_orientation(), analyze_site(). Covers known cities, fallbacks, edge cases (equator, southern hemisphere, zero area, clamped score).
- test_energy_engine.py (14 tests) — calculate_load() appliance aggregation, diversity factor, quantity multiplier, zero-appliance edge case. size_system() battery sizing formula verified against hand-computed expected value.
- test_engines.py (30 tests) — recommend_components() tier selection, inverter type, BOS list. estimate_cost() formula, regional multiplier, premium vs budget cost ordering. generate_roadmap() phase count, structure, safety notes, mode-specific approval language.
- test_auth.py (18 integration tests) — registration success/failure paths, login, wrong password, token refresh, /me endpoint, update name, change password (old password invalidated), logout.

5. AI-Assisted Development (AEDL)

Per the Agentic Engineering Development Lifecycle defined in the Week 1 SRS (Section 1.6), this sprint used Claude as a development assistant for:

Activity	AI Role	Human Role
Auth router & JWT utilities	Generated full implementation from spec	Code review, security check
Unit test suite (3 files)	Generated all 79 test cases	Verified against engine output; approved merge
NavBar component	Generated; added dropdown and step rail	UI/UX review; accessibility
Bug detection (bcrypt, tier test)	Caught in CI run; identified root cause; applied fix	Confirmed fix is correct; approved
This progress report	Generated complete document	Sprint data provided; accuracy verified

The human developer retains ownership of all architectural decisions, final code review, merge approval, and acceptance of every deliverable. Claude functions as a senior pair programmer and technical writer, not as an autonomous agent.

6. Sprint Retrospective

6.1 What Went Well

- The tool-calling agent architecture from Week 1 made the auth integration clean — auth sits as a separate layer without touching the engine logic.
- Deterministic engines (pure functions, no hidden state) made the test suite straightforward to write: inputs in, expected output out, assert. Zero mocking required.
- The JWT + SQLAlchemy async stack was validated against the actual Python version (3.12) in the sandbox — the bcrypt/passlib version conflict was caught and pinned before shipping.
- Frontend build pipeline (Vite) compiled 1,575 modules cleanly after adding 3 new components — no breaking changes to existing forms or dashboard.
- AI-assisted scaffold (Copilot-style): Claude generated the auth router, test files, and NavBar from spec in a single pass, with only a tier-boundary fix in one test after seeing real engine output.

6.2 What Could Be Improved

- Token blacklist (logout revocation) is not yet implemented — logout is currently client-side only. Planned for Sprint 2 / E9.
- GitHub Projects board still not configured — this task slipped from Week 2 and must be completed in the first two days of Sprint 2.
- The bcrypt/passlib version conflict indicates the requirements.txt needs more precise pinning from the start; added bcrypt==4.0.1 pin.
- Password reset flow (forgot-password email) is not yet scoped — should be added to the backlog as a Sprint 2 stretch goal.

6.3 Action Items for Sprint 2

- 1. Sprint 2, Day 1: Set up GitHub Projects board and import all open backlog items.
- 2. Sprint 2, Week 5: Add token blacklist (Redis or DB table) for proper logout revocation.
- 3. Sprint 2, Week 5: Implement CRUD operations for saved solar projects per user.
- 4. Sprint 2, Week 5–6: Scope password-reset flow as stretch goal.
- 5. Ongoing: Keep bcrypt and passlib versions pinned in requirements.txt.

7. Sprint 2 Preview (Weeks 5–6)

Based on the initial task list (docs/initial_task_list.md), Sprint 2 will focus on:

- CRUD operations — save, retrieve, rename, and delete solar consultation sessions per user
- Forms enhancement — pre-fill from a saved session, compare two sessions side-by-side
- API integration — wire in PVGIS irradiance API to replace the estimation model for real-time data
- Sprint velocity tracking — AI monitors completed vs planned tasks and suggests backlog adjustments
- Sprint 2 retrospective — AI-generated at the end of Week 6

- Full deployment config — Dockerfile, environment variable documentation, CI pipeline

Sprint 2 start date: Week 5. Supervisor review checkpoint: end of Week 5 (before API integration begins).

Appendix — Sprint 1 File Changes

New files (Sprint 1)

```
backend/app/database.py
backend/app/models.py
backend/app/auth.py
backend/app/routers/__init__.py
backend/app/routers/auth_router.py
backend/app/routers/session_router.py
backend/tests/__init__.py
backend/tests/test_site_analysis.py
backend/tests/test_energy_engine.py
backend/tests/test_engines.py
backend/tests/test_auth.py
backend/pyproject.toml
frontend/src/context/AuthContext.jsx
frontend/src/components/LoginPage.jsx
frontend/src/components/RegisterPage.jsx
frontend/src/components/NavBar.jsx
```

Modified files (Sprint 1)

```
backend/app/session_store.py  — added user_id, get_by_user(), delete()
backend/app/main.py           — auth/session routers, lifespan startup, health
version bump
backend/requirements.txt      — added auth + test deps; bcrypt pinned to 4.0.1
frontend/src/api.js           — added auth endpoints (register, login, logout, getMe,
updateMe)
frontend/src/App.jsx          — auth flow, AuthProvider, NavBar, demo mode
```